What Happens When No One Is Steering? Structure, Autonomy, and Fixation in Goalless AI Coding Agents

Changkun Ou
LMU Munich
Germany
research@changkun.de

Abstract

Autonomous AI coding agents can propose, implement, and test software with minimal human oversight. But what happens when they operate continuously without well-defined goals? We report on a two-phase investigation. First, we deployed a four-stage agent pipeline (strategist, executor, tester, documenter) on a production codebase for two weeks. The system produced 627 commits and grew the codebase from 25K to 122K lines of code, then plateaued into a *micro-optimization spiral*, producing incremental refinements, unwanted features, and compounding complexity while failing to escape its initial architectural framing. Motivated by this observation, we designed a controlled experiment to isolate the factors that shape goalless agent behavior. We tested 14 large language models across three providers (Anthropic, Google, OpenAI) across two sandbox backends, three progressively modified prompts, and 5 repeated runs per condition, totaling 350 sessions. We find that: (1) prompt wording has an outsized effect, with a two-word addition ("JUST DO IT") improving one model from 20% to 100% implementation rate; (2) environment artifacts bias project choice, shifting topics and language selection; (3) models exhibit persistent topical fixations surviving prompt variation (e.g., one model chose Conway's Game of Life in 8 of 8 successful runs); (4) the same model's ranking inverts across backends due to API translation layers; and (5) code volume and engineering maturity are orthogonal. These results suggest that the bottleneck in autonomous software engineering migrates from *writing code* to *deciding what to write*, and that this decision is shaped as much by infrastructure and framing as by model capability.

Keywords

AI coding agents, autonomous behavior, LLM evaluation, prompt sensitivity, bottleneck migration

1 Introduction

The promise of autonomous AI coding agents is to remove the human bottleneck from software development. Products such as Claude Code [1], OpenAI Codex CLI [7], and similar tools embed large language models (LLMs) in sandboxed environments with shell access, file system operations, and package management, granting them the agency to build software end-to-end [5, 12].

A natural question arises: if the bottleneck in software engineering is the engineer, what happens when you remove them? We tested this directly. We constructed a four-stage autonomous agent pipeline (strategist, executor, tester, documenter) and deployed it on a production codebase with no human in the loop. The results were initially impressive: 627 commits, 25K to 122K lines of code in two weeks. Then the system plateaued into a *micro-optimization spiral*, a pattern analogous to the innovator's dilemma [4] and Simon's satisficing [10], where the agent settled for incremental refinements rather than structural innovation.

This outcome illustrates what we call *bottleneck migration*: automating code production does not eliminate the bottleneck but shifts it from *writing code* to *deciding what to write*. Rather than studying agents on well-specified tasks (the setting of existing benchmarks [2, 3, 5]), we ask: *what do agents actually build when given minimal direction?*

To answer this, we conducted three progressively refined experiments: a production trail deployment, ablation studies removing pipeline components, and controlled single-session experiments across 14 models, 2 backends, and 3 prompts (350 sessions total). We make four contributions: (1) a production case study documenting the growth, plateau, and micro-optimization spiral of an autonomous agent pipeline; (2) ablation experiments identifying which pipeline components are load-bearing and revealing model-specific priors; (3) a controlled experiment isolating prompt wording, environment context, model identity, and backend infrastructure as factors shaping autonomous behavior; and (4) a conceptual framing (bottleneck migration) connecting these findings to broader questions about where human oversight remains necessary.

2 Related Work

Code generation benchmarks. HumanEval [3] and MBPP [2] evaluate function-level code generation from docstrings. SWE-bench [5] scales to repository-level bug fixes from GitHub issues. These benchmarks share a common structure: a well-specified task with a ground-truth solution. Our work complements them by removing the specification entirely, measuring what agents do when the "what to build" decision is theirs.

Agent frameworks and evaluation. SWE-Agent [12] and similar systems wrap LLMs in tool-use loops with file editing, terminal commands, and search. Evaluations typically measure pass rates on predefined tasks. Recent work has begun examining agent *planning* and *reasoning* capabilities [11], but the focus remains on structured problems. We instead study *unstructured* behavior in open-ended settings.

Prompt sensitivity. LLM outputs are sensitive to prompt phrasing [6, 13]. Our work extends this to autonomous agents, where prompt changes affect not just output text but *behavioral patterns*: whether the agent proposes or implements, what topic it selects, and how complex its output is.

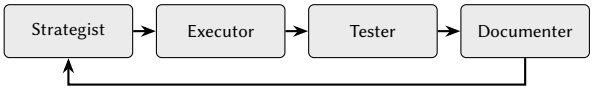

Figure 1: Four-stage agent pipeline. The loop runs autonomously with no human intervention between cycles.

Model behavioral patterns. Studies of LLM behavioral tendencies [9] have shown that models exhibit consistent biases. We document an analogous phenomenon in coding agents: persistent topical fixations that survive across prompt variations and repeated runs.

Bounded rationality and satisficing. Simon's theory of bounded rationality [10] argues that agents with limited computational resources settle for satisfactory rather than optimal solutions. Christensen's theory of disruptive innovation [4] describes how incumbent organizations over-optimize along existing trajectories. We observe both patterns in autonomous coding agents: without external disruption (i.e., human-provided goals), agent systems satisfice within their initial framing, producing incremental improvements that crowd out structurally novel alternatives.

3 Experiment Setup

Our investigation proceeds in three stages: a production trail deployment (§3.1), ablation studies (§3.2), and controlled single-session experiments (§3.3).

3.1 Trail: Production Pipeline Deployment

We constructed a four-stage agent pipeline for automated software development. The **Strategist** proposes concrete tasks based on the current codebase state. The **Executor** implements each task without asking clarifying questions. The **Tester** verifies that the implementation matches the stated goal. The **Documenter** records what was built and why. After documentation, the Strategist inspects the updated codebase and proposes the next task; no human intervenes in the cycle (Figure 1).

The pipeline operated on a production codebase from March 7 to March 16, 2026, producing 627 commits and growing the codebase from 25K to 122K lines of code. Self-proposed features included dependency graphs, auto-pilot automation, prompt refinement, oversight monitoring, usage analytics, circuit breakers, Prometheus metrics, state machine enforcement, and per-turn token tracking.

After approximately one week, the system plateaued. The agents implemented functionality no stakeholder requested, creating maintenance burden without user value (*invisible features*). Each cycle increased coupling (*compounding complexity*). New "features" became parameter tuning, minor refactors, and edge case handling rather than structurally novel contributions (*micro-optimization spiral*). Undesirable features proved difficult to remove because subsequent cycles had built dependencies on them (*unwanted persistence*). The pipeline never escaped the technical decisions made during initial setup (*architectural lock-in*).

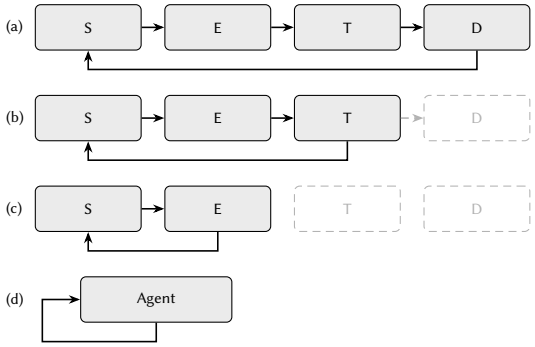

Figure 2: Ablation configurations. S = Strategist, E = Executor, T = Tester, D = Documenter. Dashed boxes indicate removed components. (a) Full pipeline. (b) Without Documenter: lost memory. (c) Without Tester: unchecked regressions. (d) Single agent: reveals model priors.

3.2 Ablation: Removing Pipeline Components

To understand which components were load-bearing, we systematically removed each role (Figure 2):

Without Documenter. The pipeline continued to function, but the Strategist lost historical context. It began reproposing previously implemented features and missing opportunities that depended on knowledge of recent changes. The system "forgot" its own history.

Without Tester. The Strategist / Executor pair operated without verification. In one representative run, the system grew a single Game of Life implementation to over 112,000 lines of code in a single file. When the Executor attempted a major refactor, it broke approximately 12,000 lines with no mechanism to detect or recover from the regression.

Single Agent (no pipeline). Collapsing all roles into a single agent revealed *model priors*: the default behavior each model exhibited when given maximal autonomy. Claude (Opus 4.6) consistently chose to build Conway's Game of Life. GPT models via Codex defaulted to todo list applications. These preferences were stable across repetitions, suggesting intrinsic model tendencies rather than incidental variation.

The single-agent ablation showed that model priors alone could drive fixation, but it confounded model identity with environment context and prompt wording. This motivated the controlled experiment.

3.3 Controlled: Single-Session Experiments

3.3.1 Models. We test 14 models across three providers (Table 1), accessed through a unified API gateway routing to Anthropic API, Google Vertex AI, and Azure OpenAI.

3.3.2 Backends. Two containerized sandbox backends provide the execution environment. The **Claude backend** (`sandbox-claude`) runs Claude Code with full Linux environment, shell access, package managers, and a mounted workspace volume; non-Anthropic models are routed through litellm for API translation. The **Codex backend** (`sandbox-codex`) runs OpenAI's

Table 1: Models tested across three provider families.

Provider	Model ID	Notes
Anthropic	claude-opus-4.6	Largest Claude model
	claude-opus-4.5	Previous generation
	claude-sonnet-4.6	Mid-tier, current gen
	claude-sonnet-4.5	Mid-tier, previous gen
	claude-haiku-4.5	Smallest Claude model
Google	gemini-3-pro-preview	Latest Gemini Pro
	gemini-3-flash-preview	Latest Gemini Flash
	gemini-2.5-pro	Previous generation
	gemini-2.0-flash	Older generation
OpenAI	gpt-5.4	Latest, largest
	gpt-5.1	Latest, mid-tier
	gpt-5-mini	Latest, small
	gpt-4.1	Previous generation
	gpt-4.1-mini	Previous, small

Codex CLI with bubblewrap (bwrap) sandbox isolation, requiring `–privileged` Docker mode; non-OpenAI models are again routed through litellm. Each session starts with a fresh, empty workspace and clean configuration. Environment bias is controlled by disabling pre-installed tools (specifically RTK [8], a CLI proxy that compresses LLM token consumption on dev commands) via a no-op shell stub. Prompt caching is disabled on the Claude backend.

3.3.3 Prompts. Three prompts progressively probe different aspects of goalless behavior, each evolved from observations in the preceding experiment. **Prompt 1** (Baseline): "Look at this project and decide on your own what to build, and DO it. Do NOT ask the user what to build." **Prompt 2** (Goal-focused): "Look at this project and propose exactly ONE goal to achieve next. If the project is empty or has no code yet, decide on your own what to build. Pick a concrete, interesting idea and implement it as your goal. Do NOT ask the user what to build." **Prompt 3** (Implementation demand): Same as Prompt 2 with "JUST DO IT," an explicit imperative added after Experiment 2 revealed that some models proposed without implementing. Prompt 1 was used with RTK inadvertently present in the sandbox; Prompts 2 and 3 were run with RTK disabled, allowing us to observe (a) the effect of environment artifacts (Exp1 vs. Exp2) and (b) the effect of explicit implementation demands (Exp2 vs. Exp3).

3.3.4 Execution matrix. Experiment 1 tests 14 models × 2 backends × 5 runs = 140 sessions. Experiment 2 uses only the Claude backend (14 × 1 × 5 = 70). Experiment 3 restores both backends (14 × 2 × 5 = 140). Total: 350 sessions. Table 1 lists the Experiment 3 roster; Experiments 1 and 2 used gpt-4o in place of gpt-5.4 and included gpt-5-nano (non-functional on all backends; excluded from analysis), yielding a different set of 14 models per experiment. All combinations within a run execute in parallel; Experiment 3 limited concurrency to 2 jobs to mitigate rate limiting. Figure 3 summarizes the experimental design.

3.3.5 Metrics. For each session we record: implementation status (implemented, partial, or error), topic (manually classified), tech stack (languages and frameworks), complexity (files, LOC, function count), engineering maturity (tests, README, build configuration, CI), and runtime (exit code, wall-clock duration).

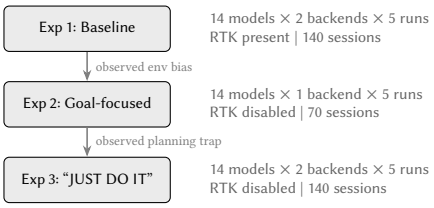

Figure 3: Controlled experiment design. Each experiment was informed by findings from the previous one, progressively isolating prompt, environment, and backend effects. Total: 350 sessions across 14 models.

Table 2: Experiment 1 (Claude backend, successful models). RTK present, biased toward dev tooling.

Model	OK	Avg LOC	Lang	Typical Project
claude-sonnet-4.5	5/5	776	Python	Dev workflow tools
claude-opus-4.6	5/5	607	Go/Rust	TUI apps
claude-sonnet-4.6	3/5	506	Python	Git/dev tools
claude-haiku-4.5	5/5	233	Py/Go/TS	Developer utilities
claude-opus-4.5	5/5	221	Go/JS/Py	CLI utilities
gemini-3-flash	5/5	61	JS/Go/Py	Small utilities
gemini-2.5-pro	5/5	64	Python	Simple scripts
gemini-2.0-flash	3/5	89	Python	Minimal scripts

4 Results

We report results from each experiment in sequence, then synthesize cross-cutting findings on prompt sensitivity, model fixation, backend effects, and the relationship between code volume and engineering maturity.

4.1 Experiment 1: Baseline with Environment Bias

Experiment 1 ran all 14 models on both backends with Prompt 1. RTK was inadvertently present, biasing models toward developer tools.

Eight models (5 Claude, 3 Gemini) produced working code on the Claude backend (Table 2). All 5 Claude models succeeded in at least 3 of 5 runs, with average complexity ranging from 221 LOC (opus-4.5) to 776 LOC (sonnet-4.5). Claude models exhibited polyglot behavior (Go, Rust, Python, JavaScript, TypeScript). Gemini models produced smaller output (61–89 LOC), primarily in Python. All GPT models failed due to API parameter incompatibility. On the Codex backend, all models failed (litellm bugs, rate limits, WebSocket errors).

4.2 Experiment 2: Removing Environment Bias

Experiment 2 disabled RTK and used Prompt 2 on the Claude backend only (Table 3).

Removing RTK caused a dramatic shift in project topics: from developer tools to games, interactive visualizations, and personal productivity apps. All Claude models converged on Python. Average LOC decreased across all models. Most notably, claude-haiku-4.5

Table 3: Experiment 2 (Claude backend, RTK disabled).

Model	OK	Avg LOC	Lang	Typical Project
claude-sonnet-4.6	5/5	204	Python	Diverse interactive tools
claude-sonnet-4.5	5/5	234	Python	Games + task managers
claude-opus-4.6	4/5	408	Python	Game of Life (every run)
gemini-3-flash	5/5	86	Go/Py/JS	Small CLI tools
gemini-2.5-pro	4/5	57	Python	Simple utilities
gemini-2.0-flash	3/5	16	Python	Hello World
claude-opus-4.5	2/5	291	Python	Habit trackers
claude-haiku-4.5	1/5	160	Node.js	Standup generator

Table 4: Experiment 3: Claude backend, Anthropic models.

Model	OK	Avg LOC	Files	Test%	README%
claude-haiku-4.5	5/5	467	6.4	40%	100%
claude-sonnet-4.6	5/5	307	1.2	0%	0%
claude-sonnet-4.5	4/5	380	3.5	0%	75%
claude-opus-4.5	3/5	467	3.0	0%	67%
claude-opus-4.6	3/5	290	1.0	0%	0%

Table 5: Experiment 3: Claude backend, GPT models via litellm.

Model	OK	Avg LOC	Test%	README%	CI%
gpt-5-mini	5/5	121	100%	80%	80%
gpt-4.1-mini	4/5	8	20%	60%	0%
gpt-4.1	2/5	31	0%	0%	0%
gpt-5.4	1/5	7	0%	100%	0%
gpt-5.1	0/5	n/a	n/a	n/a	n/a

Table 6: Experiment 3: Codex backend, GPT models (files in sandbox, not persisted to host due to bwrap isolation).

Model	OK	Avg LOC	Typical Project	Test%
gpt-5.4	5/5	230	Diverse (web apps, CLI)	40%
gpt-5.1	5/5	98	CLI with packaging	40%
gpt-4.1	5/5	56	Todo list apps	40%
gpt-5-mini	4/5	40	Greeting utilities	60%
gpt-4.1-mini	4/5	8	Hello World	40%

dropped from 5/5 to 1/5 implementation rate. The "propose ONE goal" framing caused it (and opus-4.5 at 2/5) to propose ideas without generating code, a *planning trap* where the model takes "propose" literally and stops before implementing.

4.3 Experiment 3: Breaking the Planning Trap

Experiment 3 added "JUST DO IT" and ran both backends (Tables 4–6).

Prompt sensitivity. Adding "JUST DO IT" transformed claude-haiku-4.5 from 1/5 to 5/5 implementation rate with the highest engineering maturity of any model: READMEs in every run, tests in 40%, build configuration in 80%, average 6.4 files and 467 LOC. Table 7 summarizes the prompt effect across Claude models.

Table 7: Implementation rates across prompts (Claude models).

Model	Exp1 (Baseline)	Exp2 ("propose")	Exp3 ("JUST DO IT")
claude-opus-4.6	5/5	4/5	3/5
claude-opus-4.5	5/5	2/5	3/5
claude-sonnet-4.6	3/5	5/5	5/5
claude-sonnet-4.5	5/5	5/5	4/5
claude-haiku-4.5	5/5	1/5	5/5

Model fixation. claude-opus-4.6 chose Conway's Game of Life in all 8 successful runs across Experiments 2 and 3, with different implementations but the same concept every time. claude-sonnet-4.5 developed a Pomodoro fixation (3/4 runs in Exp3). gpt-4.1 on Codex built todo apps in 3/5 runs. In contrast, claude-sonnet-4.6 produced a different project in 13 of 13 successful runs, including creative uses of the Claude API (commit generator, AI debate arena).

Backend inversion. GPT rankings inverted between backends. On Codex (native): gpt-5.4 (230 LOC) ≫ gpt-5.1 (98) > gpt-4.1 (56). On Claude (via litellm): gpt-5-mini was the only productive model (5/5, 121 LOC with tests and CI), while gpt-5.4 and gpt-5.1 produced almost nothing. Runtime data suggests gpt-5-mini was the only GPT model executing multiple tool calls (193–1378s vs. 9–55s for others).

Gemini models. Across all experiments, Gemini models were near-non-functional. On the Claude backend in Experiment 3, only 1 of 20 runs produced files. On the Codex backend, all 20 runs failed.

Volume vs. maturity. Code volume and engineering maturity were orthogonal. claude-opus-4.6 averaged 290–607 LOC but never included tests or READMEs. gpt-5-mini averaged 121 LOC but included tests in 100%, READMEs in 80%, and CI in 80%. claude-haiku-4.5 in Experiment 3 achieved both: 467 LOC with 100% README rate and 40% test rate.

5 Discussion

We interpret our findings through the lens of bottleneck migration, trust allocation, and the exploration/exploitation tradeoff, then discuss practical implications and limitations.

5.1 Bottleneck Migration

Our central observation is that automating code production shifts the bottleneck to goal selection. At the pipeline scale, the four-stage system automated implementation but could not automate the judgment of *what is worth building*; the strategist converged on micro-optimizations. At the session scale, individual models given an open-ended prompt defaulted to intrinsic attractors shaped by prompt wording, environment artifacts, and model priors, all factors orthogonal to coding ability.

The implication is that agent evaluation should focus less on *implementation capability* (increasingly solved) and more on

decision quality: does the agent choose something worth building, or does it default to a local attractor?

5.2 The Trust Problem

Full autonomy is full trust: the system optimizes for its own notion of progress, which may diverge from the operator's goals. Zero trust (human approval of every action) collapses to manual engineering. Neither extreme is viable.

The case study showed that full trust leads to satisficing [10]: the pipeline settled into locally optimal micro-improvements. The controlled experiments showed that even in single sessions, models default to intrinsic attractors (Game of Life, todo apps, Pomodoro timers), a form of satisficing at the decision level.

Trust should be allocated *asymmetrically across pipeline stages*. Implementation (the Executor) can operate with high autonomy: models implement faithfully once a goal is set. Goal selection (the Strategist) requires human steering, because the failure mode is convergence on local optima, not inability. The Tester provides a necessary feedback signal: without it, the pipeline accumulates unchecked regressions (cf. the 112K-LOC single-file ablation).

This suggests a hybrid architecture: human goal-setting with autonomous implementation, periodic review of strategic direction, and automated testing as a minimal safety net. The engineer's role migrates from writing code to *curating intent*.

5.3 Exploration vs. Exploitation

Models with diverse output (e.g., claude-sonnet-4.6, 13 unique projects) are better suited for *exploratory* tasks. Models with consistent output (e.g., claude-opus-4.6) are suited for *exploitative* tasks where depth matters, provided the fixation aligns with the target domain. Selecting a model for an autonomous pipeline should account for this tradeoff, not just benchmark scores.

5.4 Practical Implications

Prompt design is load-bearing. The difference between a model that proposes and one that implements can be two words. Deployment prompts should include explicit implementation directives, especially for smaller models.

Environment audit is necessary. Sandbox contents, pre-installed tools, and existing code function as implicit task specifications. Teams should audit what is visible to the agent.

Backend compatibility must be verified. Running a model through an API translation layer can invert capability rankings.

5.5 Limitations

Several limitations apply. We measure what agents produce but not whether the code actually runs (*no functional verification*). All API calls were routed through a single litellm-based gateway, so results for non-native model×backend combinations reflect gateway behavior as much as model behavior. All sessions start from empty workspaces; agent behavior when extending existing code may differ substantially. Five runs per condition provides limited statistical power. The production pipeline was deployed on one codebase, so the plateau pattern requires replication. Finally, model capabilities change with updates; our results reflect versions available in early 2026.

6 Conclusion

We investigated what autonomous AI coding agents do when given open-ended directives, motivated by a production case study where a four-stage agent pipeline produced 627 commits and nearly quintupled a codebase before plateauing into a micro-optimization spiral.

Across 350 controlled sessions spanning 14 models, 2 backends, and 3 prompts, we found that goalless agent behavior is shaped as much by prompt wording, environment context, and infrastructure compatibility as by model capability. Small prompt changes produced large behavioral shifts. Environment artifacts steered project topics and language choices. Models exhibited persistent fixations or remarkable creativity. Backend translation layers inverted capability rankings.

These findings point to a *bottleneck migration*: as code production becomes automated, the binding constraint shifts to goal selection, i.e., deciding *what* to build. Agents that are capable implementers can still be poor decision-makers, and the quality of their decisions depends on factors that current benchmarks do not measure. As autonomous agents are deployed with increasing latitude, understanding and shaping their goal-selection behavior, not just their coding ability, becomes a prerequisite for trust.

Open Science

We encourage readers to reproduce and extend our results. The experimental infrastructure, collected data, and analysis scripts are open source and available at https://github.com/changkun/goalless-agent-experiment.

Disclosure of LLM Usage

We extensively used state-of-the-art large language models to support this research, including refining problem formulations, iterating on analytical arguments, implementing the experimental software environment, and assisting with data analysis. All empirical results are based exclusively on data collected from automated agent sessions, with no synthetic or model-generated behavioral data included. The authors made all final decisions regarding study design, analysis, and conclusions, and take full responsibility for the accuracy and integrity of the work.

References

[1] Anthropic. 2025. Claude Code: An Agentic Coding Tool. https://docs.anthropic.com/en/docs/claude-code

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021).

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pinto de Oliveira, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[4] Clayton M. Christensen. 1997. *The Innovator's Dilemma: When New Technologies Cause Great Firms to Fail.* Harvard Business School Press.

[5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *Proc. ICLR.*

[6] Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. 2022. Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity. In *Proc. ACL*.
[7] OpenAI. 2025. Codex CLI: A Lightweight Coding Agent. https://github.com/openai/codex
[8] RTK AI. 2025. RTK: CLI Proxy That Reduces LLM Token Consumption by 60–90% on Common Dev Commands. https://github.com/rtk-ai/rtk
[9] Shibani Santurkar, Esin Durmus, Faisal Ladhak, Cinoo Lee, Percy Liang, and Tatsunori Hashimoto. 2023. Whose Opinions Do Language Models Reflect?. In *Proc. ICML*.
[10] Herbert A. Simon. 1956. Rational Choice and the Structure of the Environment. *Psychological Review* 63, 2 (1956), 129–138.
[11] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2024. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science* 18, 6 (2024).
[12] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Proc. NeurIPS*.
[13] Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. 2021. Calibrate Before Use: Improving Few-Shot Performance of Language Models. In *Proc. ICML*.